RA2311047010044 38

INDEX

NAME K.D.D.SaiAbhiram SUBJECT Deep learning lab obeb

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Analyze and exploring the deep learning platforms	24-09-25	
2		Implement a classifier using open-source data set	7/8/25	
3.		study of the classifiers with respect to statistical parameters	7/8/25	
4.		Build a simple feed forward neural network to recognize hand written characters	14/8/25	
5.		Study of activation function and its role	28-8-25	
6.		Implement gradient descent & back propagation in deep neural network.	09-9-25	
7.		Build a CNN model to classify cats & dogs images	16-9-25	
8.		Experiment using LSTM		
9.		Build a RNN		
10		Perform Compression on MNIST data set using Autoencoder		
11.		to implement experiment using variational Autoencoder		
12.		Implement DCGAN to generate colour images	27/10	
13.		understanding the architecture of Pretrained model	27/10	
14.		Pre-trained CNN model as feature extractor using TLM		
15.		Implement a YOLO model to detect objects		

complete

8. Experiment using LSTM

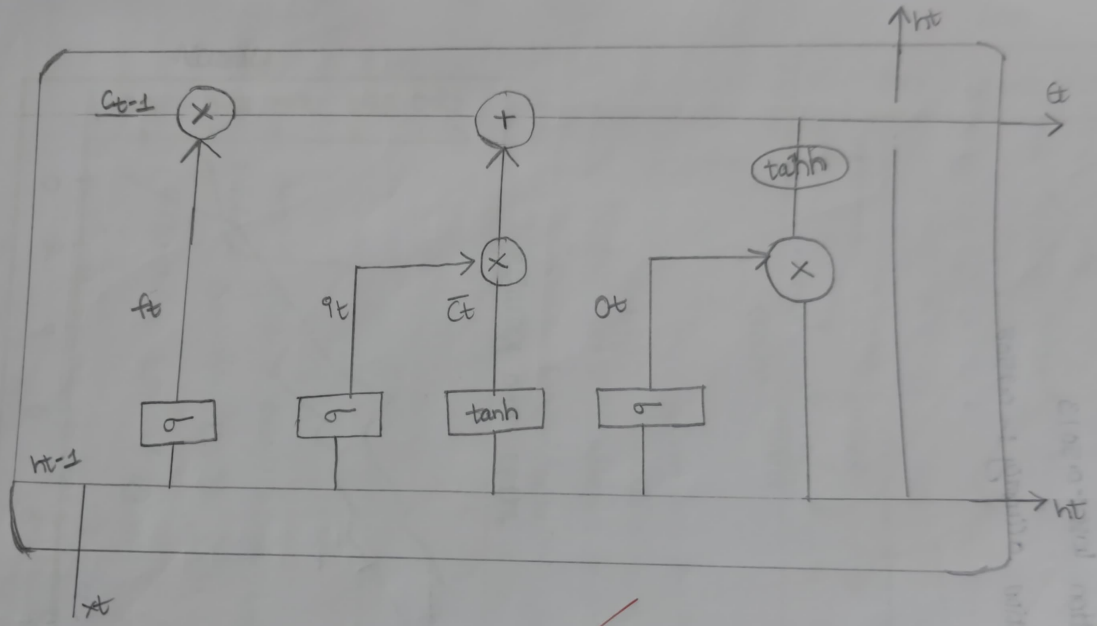
Aim:- to implement a long short-term memory (LSTM) neural network in PyTorch and analyze its performance on a predictive task.

Objectives:-

1. to understand the working mechanism of LSTM in layers of RNN's.
2. to learn and how to prepare sequential data for training an LSTM.
3. to train the model to predict the next element in a numerical
4. to evaluate the learning model performance using loss metrics and outputs.

Pseudo Code :-

1. Import necessary libraries
   (torch, torch.nn,)
2. Prepare sequential (or) open data source
3. LSTM Model class.
4. Define forward Propagation
5. loss function (or) Cross entropy loss of classification
6. optimizer SGD
7. train the model
   for each epoch
   * zero gradient
   * forward pass
   * Backward pass
   * update parameters
8. Evaluate model on test data
9. display predicted vs actual values.



Long short term memory

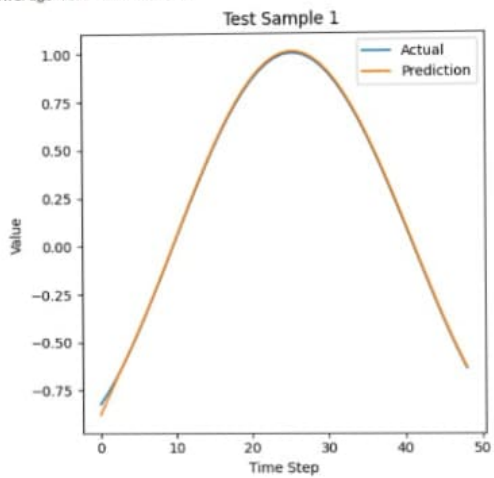
```python
# 7. Print the average test loss
print(f'Average Test Loss: {average_test_loss:.4f}')

# 8. Optionally, visualize some predictions from the test set
# Select a few samples to plot
num_plots = 3
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 5))
for i in range(num_plots):
    plt.subplot(1, num_plots, i + 1)
    # Assuming each batch has at least 'i' samples and each sample has enough sequence length
    plt.plot(all_targets[0][i, :], label='Actual')
    plt.plot(all_predictions[0][i, :], label='Prediction')
    plt.title(f'Test Sample {i+1}')
    plt.xlabel('Time Step')
    plt.ylabel('Value')
    plt.legend()

plt.tight_layout()
plt.show()
```

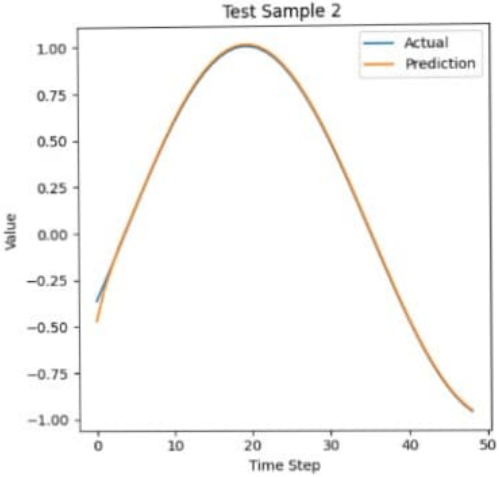
```
# 1. Generate a separate test dataset
num_samples_test = 200
sine_data_test = generate_sine_wave(seq_length, num_samples_test)

# 2. Create a TimeSeriesDataset and DataLoader for test data
test_dataset = TimeSeriesDataset(sine_data_test)
test_dataloader = DataLoader(test_dataset, batch_size=32, shuffle=False) # Shuffle is False for consistent evaluation

# 3. Set the model to evaluation mode
model.eval()

# 4. Disable gradient calculations
with torch.no_grad():
    test_loss = 0
    num_batches_test = 0
    all_predictions = []
    all_targets = []

    # 5. Iterate through the test dataloader
    for inputs, targets in test_dataloader:
        # Get the inputs and targets.
        # Reshape the inputs
        inputs = inputs.unsqueeze(-1) # Adds a dimension of size 1 at the end

        # Pass the inputs through the model to get the outputs
        outputs = model(inputs)

        # Reshape the targets to match the outputs
        targets = targets.unsqueeze(-1) # Adds a dimension of size 1 at the end

        # Calculate the loss
        loss = criterion(outputs, targets)
```

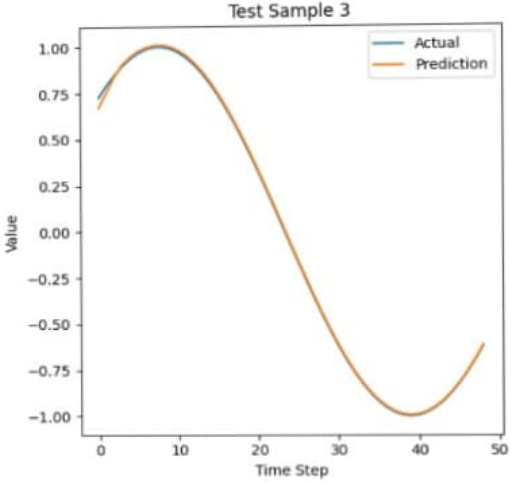
[ ]

```python
import torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, output_size):
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.linear = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # Initializing the hidden and cell states with zeros is optional for this simple model,
        # but it's good practice for more complex scenarios.
        # h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)
        # c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)

        # Forward pass through LSTM
        out, _ = self.lstm(x) # out: tensor of shape (batch_size, seq_length, hidden_size)

        # Apply the linear layer to the output of each time step
        out = self.linear(out) # out: tensor of shape (batch_size, seq_length, output_size)

        return out

# Instantiate the model
input_size = 1  # Univariate time series
hidden_size = 64 # Example hidden size
num_layers = 2  # Example number of layers
output_size = 1 # Predicting the next value

model = LSTMModel(input_size, hidden_size, num_layers, output_size)
```

Average Test Loss: 0.0002







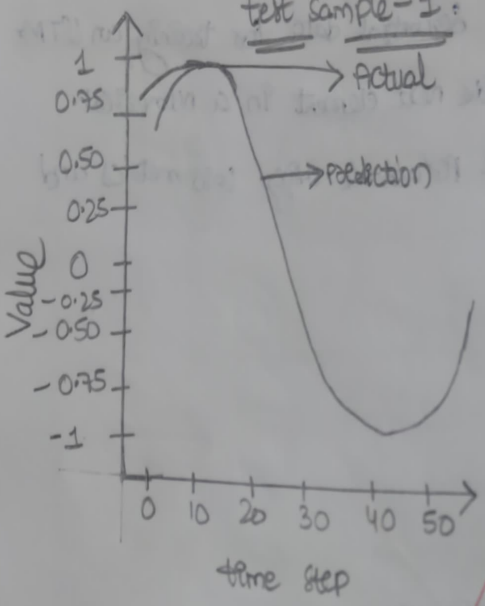
Output

Epoch [10/100] ; loss= 0.0004
Epoch [20/100] ; loss= 0.0004
Epoch [30/100] ; loss= 0.0002
Epoch [40/100] ; loss= 0.0002
Epoch [50/100] ; loss= 0.0002

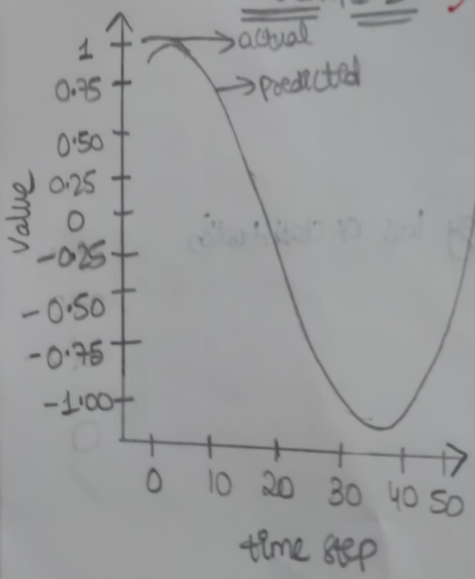
Epoch [60/100] loss= 0.0002
Epoch [70/100] loss= 0.0001
Epoch [80/100] loss= 0.0001
Epoch [90/100] loss= 0.0001
Epoch [100/100] loss= 0.0

Average test loss :- 0.0002

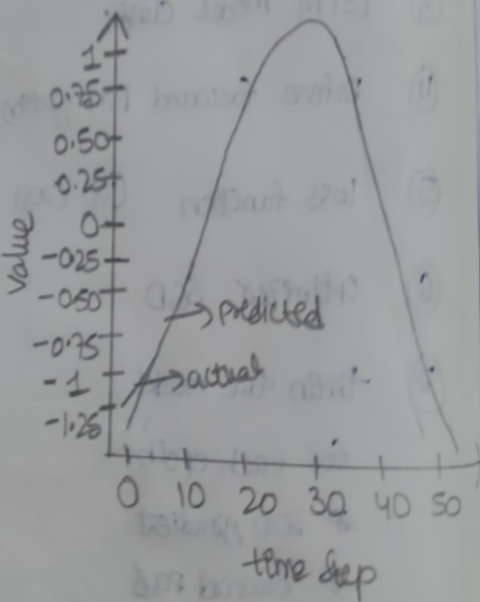
test sample-1 :



test sample-2



test sample-3



Observations

* the loss gradually decreased with epochs indicating that the LSTM is learning sequence pattern
* the model accurately predicts the next value in the numeric sequence.

Result

* The LSTM network was successfully implemented using pytorch.
* Average test loss is 0.0002.